



CO3204 Software Engineering Project Interim Report

Automatic Classification of medical X ray images

Alex George

November 2025

School of Computing and Mathematical Sciences

University of Leicester

Contents

1 Aims and Objectives.....	3
1.1 Aims.....	3
1.2 Motivation For The Project.....	3
1.3 Objectives	4
2 Literature Review	5
2.1 Augmentation Methods.....	5
2.2 CNN Architectures and their Use in Image Classification	6
2.3 Evaluation Methods.....	7
3. Requirements Analysis	8
3.1 Functional Requirements.....	8
3.2 Non Functional Requirements	8
4. Timeline.....	9
5. Software Architecture and Design.....	11
5.1 The Dataset.....	11
5.2 Code Libraries and Tools to be used.....	11
5.2.1 Image Preprocessing	11
5.2.2 Deep Learning Code	11
5.3 Architecture	11
6. References.....	13

1 Aims and Objectives

1.1 Aims

The overall aim of this project, is to design, develop, and implement a robust CNN model to accurately classify medical images in the ChexPert V1 Small dataset, despite significant class imbalance being present in the dataset.

The CNN I will create will also be relatively lightweight, due to the computational constraints that will be present in development, but also, due to the application this project could have in hospitals. Hospitals are less likely to have very powerful supercomputers, comparable to those used by researchers, and industry-leading firms, and so they require a solution that is both lightweight, and effective at classifying images, and so I will aim to make the network relatively shallow, which coincides with it being easier to train.

To ensure a robust, imbalance-resistant CNN is created the project will strive to achieve the following:

1. Create a baseline model, and evaluate it using metrics such as accuracy, recall, precision and F1 Score
2. Evaluate and compare different augmentation methods (such as flipping, rotation, cropping etc.)

1.2 Motivation For The Project

Classification is a method in which images are categorized into distinct categories. This means that objects, such as medical x-ray images, are grouped into pre-existing categories, or labels. For example, when given an image of a Chest X-ray, classification aims to predict whether the x-ray provided depicts someone suffering with Pneumonia, or not.

One of the biggest problems, when using Neural Networks to classify medical x-ray images, is that of class imbalance. This is when there are many more instances (images) in one class, compared to another. This can lead to the model performing badly on the more rare classes, which is a big dilemma in medical imaging, as certain conditions are studied, x-rayed and, encountered more often than others, leading to a natural imbalance in the number of x-rays showing one condition, such as pneumonia, compared to another, such as chest lesions.

This project is especially valuable when trying to diagnose certain medical conditions. For instance, the NHS state that 84-94% of imaging studies are completed within 2 weeks [1], which could potentially be cut down to a matter of days, as the CNN will be able to predict the class of an x-ray quicker than compared to a radiologist doing the same. The Convolutional Neural Network I create can also be useful for reducing the chance of misdiagnosis. [2], found that Deep Neural Networks were more accurate than humans at detecting the sexual orientation of a human from pictures of their face, which, while not directly related to x ray images, suggests that the use of CNNs to classify chest x-ray images, could surpass humans in terms of accuracy. This can lead to more accurate diagnoses, which ultimately lead to better patient survival rates, and satisfaction.

1.3 Objectives

1. **Data Preparation:** Sample the chexpert Dataset down to between 1000 and 5000 images (as of yet the exact number is undecided), while keeping the imbalance,
2. **Handle Labels:** Uncertainty labels are present, meaning some labels are not confirmed to be a part of the image, meaning I can handle this by converting any uncertain labels to negatives, to ensure a simpler, classification
3. **Baseline CNN Architecture Implementation:** Design a Baseline CNN, with 2 convolution layers with 3x3 filters and ReLu Activation, 2 max pooling layers, and a fully connected layer to perform the final classification
4. **Baseline CNN Evaluation:** Evaluate the baseline model in terms of Accuracy, Recall, Precision, and F1 Score
5. **Implementation of different data augmentation methods:** Iteratively introduce augmentation methods, such as Flipping, Rotations, Cropping, and colour space augmentations
6. **Evaluation of different data augmentation methods:** Evaluate different augmentation methods against each other, and a baseline CNN with no augmentation methods, using accuracy, precision, recall, and F1 Score
7. **Comparison of different augmentation methods, and selection of best performing methods:** Compare the performance of different augmentation methods, and decide on a combination that performs the best, within computational constraints
8. **Interface Design and development:** Design an easy to use interface, allowing the user to drag and drop an unseen image after training, which will then be classified by the model

2 Literature Review

Throughout the first few weeks of the project, I have spent time reading papers and journal articles about the domain in which this project operates. I have decided to organize my reading, into 3 sections: Augmentation, different CNNs and their use in image classification, and methods of evaluating the performance of the model. The following sections will summarize the findings from my reading so far.

2.1 Augmentation Methods

After comparing different approaches to classification, my research shifted focus to focussing on different augmentation methods, and how they affect performance. Wang and Perez [3] conducted an experiment to compare different augmentation techniques, and investigate the impact they had on a model's performance. They did this by using a dataset called Tiny Imagenet 200, on which they would first train their CNN to perform a rudimentary classification and then retrain it after using traditional augmentation techniques, and then again using CycleGANs, to augment the data. They sampled the dataset, by taking 500 images each of the "Dog", "Cat", and "Goldfish" classes, and trained their network with 10 epochs. This paper also highlights the effect of inter-class similarity, and intra-class variability, which are demonstrated in the classes chosen, as cats and dogs, are more similar to each other than dogs and goldfish. When classifying dogs vs goldfish, they found that using a neural network results in the best performance, as it had an accuracy of 0.775, whereas when classifying dogs vs cats – an application in which there is more inter-class similarity compared to goldfishes – they found that traditional methods had a higher accuracy, of 0.915.

This suggests that in my project classifying chest x-rays, traditional methods of augmentation may perform better than deep learning methods, as there is more inter-class similarity, due to all objects in the chexpert dataset being chest x-rays

[4] took a different approach to classifying datasets which are imbalanced – oversampling, and under sampling. They investigated this by comparing 17 datasets, in a binary classification problem, across different levels of imbalance by oversampling, and under sampling the classes, and evaluating the performance of their model. Similar to the ChexPert dataset to be used in my project, they found that in datasets where there is a high level of class imbalance, oversampling was more effective for minimising the effect of class imbalance than under sampling, though they noted that undersampling was still better than not addressing the imbalance at all.

For my project, this presents an opportunity to combine both augmentation, to get a more varied dataset, and oversampling, to further reduce class imbalance, and improve performance of the model.

2.2 CNN Architectures and their Use in Image Classification

I also researched different CNNs and how they were used across different contexts, and how different types of CNNs, with different layers, being trained on different images, etc. affected the performance, to better understand how to best design my network to accurately classify chest X-Ray Images.

One of the reasons I decided to go with a deep learning approach is that CNNs are often more accurate than other classification approaches, especially due to the fact that CNNs can extract features automatically, whereas other approaches require manual feature extraction using, for example Local Binary Pattern (LBP) feature extraction. [5] conducted an experiment comparing CNNs, on a lung image dataset, with other methods such as LBP and SIFT, and found that CNNs had the best performance across the dataset, with a large margin to LBP and SIFT. This solidifies my choice, as lung sample images are, like Chest X-Rays, relatively limited, and their solution was also not problem specific, meaning that it can be applied to many different domains, potentially including multi-class classification problems, like that in the ChexPert dataset in my problem. Their findings also support my decision to use a lightweight CNN, as their architecture, which used a convolution layer with a 7x7 filter, and 16 channels, with max pooling and a 2x2 filter size, and a 100-50-5 neuron fully connected layer, performed as well as models with multiple layers, meaning certain well regarded networks, such as ResNet, may be unnecessarily complex – especially if this is to be used in hospitals with limited computing resources.

Their findings are supported further by [6], where they compared 3 different CNN architectures – including their own 6 layer architecture, and ResNet – a well known and well-regarded neural network. They tested on images of people ranging from 16 to 77 years old. They found, interestingly, that the 6 layer CNN had a higher mean success in terms of one-off success, than ResNet. ResNet had a one-off success of 92.96%, compared to the 6 layer CNN which had a one-off success of 95.82%. The ResNet layer had 12 more layers (18 in total) compared to the 6 layer architecture, and this supports the interpretations of [5] that a larger network, is not necessarily more accurate. They also used some of the traditional augmentation techniques such as those in [3], such as rotation, emphasizing the idea that a relatively lightweight CNN (at least when compared to the more established Nets like ResNet), combined with traditional augmentation techniques can still provide excellent results. This is especially important in my project, as due to computational constraints, and limits in the performance of computers I have available, a lightweight solution will be prioritized.

A problem that has not been covered much by these previous papers, is that of intra-class variance and inter-class similarity. [7] has proposed a solution to this, which is to use a different loss function in the final layers of the CNN. [7] proposed a triplet loss function. This function works by taking an image, as the “anchor”, and selecting a positive image (from the same class as the anchor), and a negative image (from a different class as the anchor). It aims to minimize weights such that the distance between the anchor and positive class is minimized, and the distance between the anchor and negative class is maximized [8]. They also proposed using a hybrid model, which includes the use of X Means clustering, which starts with a minimum amount of clusters, and increases this number to find the optimum number of clusters, to aid classification [9]. They found that triplet loss and x-means clustering both performed better than standard classification, in terms of F1 Score values, though clustering performed better than triplet loss. In my project, I could potentially look into implementing both methods, with

the CNN being used for feature extraction, and then X-Means to perform the classification, though this may be outside the scope of this project.

A factor which influences training time, and overall how lightweight a model is, as well as its performance, which has not been covered in great detail by the previous paper is about choosing the size of the kernel. Kernels are small matrices which “slide” over an image, to extract features such as edges, textures or corners [10]. A study by [11] aimed to investigate the effects that a different number of filters and size of filters had on the performance of a CNN. They tested this by varying the number of filters from 32, to 64 and finally 128 3x3 filters, with a pooling layer of size 2x2, and a stride of 2. They also changed the size of the filter, to test a 7x7 filter, with the same overall network architecture. Interestingly, they found that when using 3x3 filters, 32 filters performed better than 64, suggesting that less filters makes the model less prone to overfitting, which is crucial in my project, as there is heavy imbalance, and so the model could learn majority classes too well. The finding that training time increases with the size of the filters, has convinced me, taking into account the hardware constraints, and aim of the project to provide a lightweight CNN, that more filters, with smaller dimensions may strike a nice balance between effective feature extraction and practicality.

While [5] and [6] seem to suggest that shallower networks can perform as well as deeper networks, this is debated by [12], as they used a transformer with a CNN, as they found that in some medical conditions there may be interdependency between anatomical structures, and as such, different labels. They used a CNN to extract features from the images, and send these, along with label embeddings (which are labels represented as dense vectors, allowing similarities between labels to be better seen by the model), to a transformer, and after this fuse the results together before classification. They found that their “CTrans CNN” had the highest AUC score on 7 out of 11 categories in the Chest X-Ray 14 dataset. In contrast to [5] and [6] though, they found that, specifically in this medical context, ResNet 50 performed better than ResNet 34, suggesting deeper networks may be more beneficial for medical imaging as it can extract more subtle features. This is something I must consider in my project, as I must make the network deep enough to detect these differences, while also not being so deep that it is impractical on less powerful machines.

2.3 Evaluation Methods

There are many evaluation metrics which can be used to evaluate model performance. Which metrics are chosen must be carefully considered, as methods like accuracy, while okay for simple classification problems, when there is as severe imbalance as in the ChexPert dataset, it can lead to the model just predicting the majority class each time, which would cause a high accuracy score (as accuracy is the proportion of all images in the dataset the model predicted correctly), even if the model didn’t predict many of the minority classes correctly.

Precision is the number of true positives, divided by the sum of the true positives, and false positives – the proportion of positive predictions the model got correct. This is better than accuracy, as it only considers the positive class, meaning it is less sensitive to imbalanced datasets.

Recall is another method of evaluating the performance of my model, which is the number of true positives, divided by the number of actual positive instances – meaning it is the proportion of the actual positive instances, that were predicted correctly by the model. In medical imaging, ensuring that any positive class – such as a person having Pneumonia – is correctly predicted is crucial, so I will consider prioritizing Recall as a key metric to evaluate the performance of my model.

F1 Score, is the harmonic mean of precision and recall, meaning it offers some of the same advantages of precision and recall, and essentially allows them to be represented in one value, for which reason I have chosen to use it as a key metric for evaluating the performance of the model, with different techniques to handle class imbalance.

3. Requirements Analysis

3.1 Functional Requirements

- FR1: The model must be able to train on the ChexPert dataset
- FR2: The model must be able to allow you to change the augmentation type that is being evaluated
- FR3: The user should be able to provide an image to the system to be classified after training
- FR4: The system should be able to support PNGs
- FR5: The system should perform normalization of the images
- FR6: The system must be able to evaluate the augmentation methods with appropriate evaluation metrics: accuracy, recall, precision f1 score etc
- FR7: The system should display an error for invalid file formats, or errors in image processing and handle these gracefully

3.2 Non Functional Requirements

3. NF1: The system should have an accuracy
4. NF2: New unseen images should be able to be classified within 10 seconds
5. NF3: The system should ensure that the original images, are not altered themselves, and that any augmentations are completed on a copy of the image
6. NF4: The prediction outputted must be displayed clearly, as a percentage
7. NF5: The system should have a disclaimer that states that these predictions must not be used on their own, and that expert advice is still important
8. NF6: The model should be able to load within 10 seconds.

4. Timeline

Key: Semester 1, Semester 2

Task	Start	End	Status
come up with my project Idea	3.10.2025	6.10.2025	Completed
Research class imbalance, existing CNN implementations, current research on methods to mitigate imbalance and their findings	6.10.2025	20.11.2025	Completed
Write and Submit Interim Report	10.11.2025	20.11.2025	In Progress
Set up the development environment	20.11.2025	20.11.2025	In Progress
Research open CV and Pytorch for image processing and creating the model	13.11.2025	25.11.2025	In Progress
Load the ChexPert V1 Small Dataset into the environment, and handle label assignment, and any missing labels if needed	26.11.2025	29.11.2025	Not Started
Normalize the pixel values of images, so all experiments are controlled and fair	29.11.2025	30.11.2025	Not Started
Create the baseline model of 2 convolution layers and no augmentation methods	30.11.2025	6.12.2025	Not Started
Evaluate the model with accuracy, precision, recall and f1 score	6.12.2025	10.12.2025	Not Started
Implement Simple Augmentation methods, such as	11.12.2025	18.12.2025	Not Started

cropping, flipping etc.			
Evaluate these methods, with metrics, and record	19.12.2025	25.12.2025	Not Started
Research more complex ways of dealing with imbalance, like weighted loss functions, and ways to handle intra-class variability and inter-class similarity like triplet loss	28.12.2025	1.1.2026	Not Started
Implement weighted loss functions into the model	2.1.2026	10.1.2026	Not Started
Evaluate this model and compare to traditional augmentation	11.1.2026	20.1.2026	Not Started
Connect The Model logic to a UI which is simple to use	22.1.2026	24.2.2026	Not Started
Interview Prep and attend	9.2.2026	20.2.2026	Not Started
Test UI and classification with unseen images	28.2.2026	6.3.2026	Not Started
Write dissertation	6.3.2026	7.4.2026	Not Started
Refine and improve the dissertation	8.4.2026	24.4.2026	Not Started
Create and attend Mini Viva with results	6.3.2026	Whenever the viva is meant to be done	Not Started
Check over and submit report	24.4.2026	27.4.2026	Not Started

In this plan, each substantial task has been given 1-2 weeks to be completed, with bigger or more important tasks given longer. I have also structured my planning in a way that I will be able to complete all necessary development and evaluation, testing etc. at least a month before the deadline for the project, so that I can have as much time as possible to write, and perfect the dissertation, and presentation. Many of these tasks, such as setting up the environment, and loading the data, will likely be done much quicker than set out here, but I have set out more time than needed, in case there are any unexpected delays.

5. Software Architecture and Design

5.1 The Dataset

The dataset that I have chosen for this project is the ChexPert V1 Small dataset, which can be found [here](#). This is a well-known, down-sampled version of the original ChexPert dataset, created by the Stanford ML Group. I chose this dataset as I believe it is a good example of the imbalance that can be present in a chest x-ray dataset. For example, the number of instances in the “no findings” class, is much higher than the number of instances in the “Pneumonia” class. This severe imbalance, added with the fact that it is a multilabel, multiclass dataset, accurately show the reality that class imbalance is a huge problem in medical image classification, and a lightweight, but still effective solution to mitigate the class imbalance has not been extensively investigated.

5.2 Code Libraries and Tools to be used

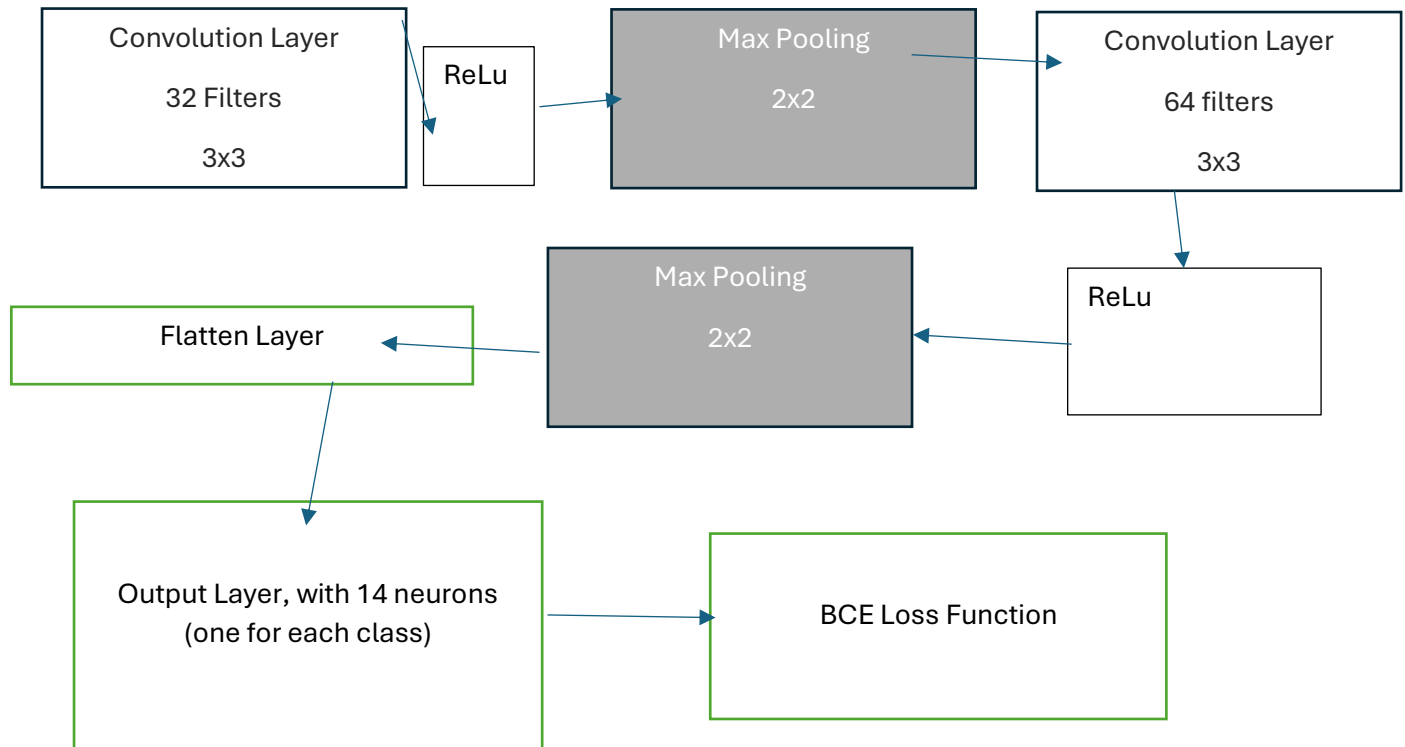
5.2.1 Image Preprocessing

For this project, I have decided to use “OpenCV-python”, for the image preprocessing, which will include normalizing the images, and certain traditional augmentation methods, such as rotations, and flipping. The decision to use OpenCV is due to the fact that OpenCV is well-regarded in the python community for being a good, well-documented library for image preprocessing, meaning that there is sufficient documentation available, if I get stuck with any aspect of the coding, for me to use to figure my way out of the problem. OpenCV, and more in general, python, has been chosen as throughout various personal projects, and modules completed during my degree, have made me more comfortable with Python. It is also the “standard” for Machine Learning projects, and papers discussing implementations often use python also, meaning there are many references, and documentation that I can look at.

5.2.2 Deep Learning Code

For the project, I have decided to use Pytorch, because, as shown in [13], PyTorch is more “pythonic”, meaning it will likely be easier to get started with, and build a simple baseline model, which I can test different augmentation methods on. [13] also shows that PyTorch is more performant than TensorFlow, as it can use asynchronous execution, which is especially important as one focus on my project is to make a small, relatively lightweight model, so a deep learning library which can improve performance is welcome.

5.3 Architecture



The diagram above shows the high level “pipeline” for my project. The images will be loaded, and, in experiments besides the baseline, will be augmented, with different techniques. This will then be passed to the CNN. First, the image will go through a convolution layer, with 32 3x3 kernels, and then the system will compute ReLu activations for each image. Then, it will pass through another convolution layer, this time with 64 3x3 filters, to extract more subtle and complex features, while still not overloading with complexity to ensure the model is relatively lightweight. Then there is an output layer, with 14 neurons, one for each class in the dataset. Finally, when training, the model will compute a binary cross-entropy loss, which allows it to fine tune predictions. Finally, we will evaluate the performance of the model with metrics such as accuracy (though, in this multilabel, multiclass dataset, this should not be trusted), recall, precision and F1 Score.

6. References

- [1] NHS England, “NHS England» Diagnostic imaging reporting turnaround times,” [www.england.nhs.uk](https://www.england.nhs.uk/long-read/diagnostic-imaging-reporting-turnaround-times/), 2023. <https://www.england.nhs.uk/long-read/diagnostic-imaging-reporting-turnaround-times/>
- [2] Y. Wang and M. Kosinski, “Deep neural networks are more accurate than humans at detecting sexual orientation from facial images.,” *Journal of Personality and Social Psychology*, vol. 114, no. 2, pp. 246–257, Feb. 2018, doi: <https://doi.org/10.1037/pspa0000098>.
- [3] Q. Li, W. Cai, X. Wang, Y. Zhou, D. Feng, and M. Chen, “Medical Image Classification with Convolutional Neural Network.” Accessed: Nov. 11, 2025. [Online]. Available: <https://meichen06.wordpress.com/wp-content/uploads/2017/03/medical-image-classification-with-convolutional-neural-network.pdf>
- [4] V. García, J. S. Sánchez, and R. A. Mollineda, “On the effectiveness of preprocessing methods when dealing with different levels of class imbalance,” *Knowledge-Based Systems*, vol. 25, no. 1, pp. 13–21, Feb. 2012, doi: <https://doi.org/10.1016/j.knosys.2011.06.013>.
- [5] Q. Li, W. Cai, X. Wang, Y. Zhou, D. Feng, and M. Chen, “Medical Image Classification with Convolutional Neural Network.” Accessed: Nov. 11, 2025. [Online]. Available: <https://meichen06.wordpress.com/wp-content/uploads/2017/03/medical-image-classification-with-convolutional-neural-network.pdf>
- [6] M. Fatih Aydogdu, V. Celik, and M. F. Demirci, “Comparison of Three Different CNN Architectures for Age Classification,” pp. 372–377, Jan. 2017, doi: <https://doi.org/10.1109/icsc.2017.61>
- [7] A. Venkataramanan, M. Laviale, C. Figus, P. Usseglio-Polatera, and C. Pradalier, “Tackling Inter-Class Similarity and Intra-Class Variance for Microscopic Image-based Classification,” *arXiv.org*, 2021. <https://arxiv.org/abs/2109.11891> (accessed May 23, 2025).
- [8] S. Reddy, “The intuition of Triplet Loss,” *Analytics Vidhya*, Nov. 02, 2021. <https://medium.com/analytics-vidhya/triplet-loss-b9da35be21b8>
- [9] “X-Means — Meteoinfo 3.6 documentation,” *Meteothink.org*, 2023. <http://meteothink.org/examples/miml/cluster/xmeans.html> (accessed Nov. 11, 2025).
- [10] GeeksforGeeks, “Kernels (Filters) in convolutional neural network,” *GeeksforGeeks*, Jun. 26, 2024. <https://www.geeksforgeeks.org/deep-learning/kernels-filters-in-convolutional-neural-network/>
- [11] “(PDF) The Impact of Filter Size and Number of Filters on Classification Accuracy in CNN,” *www.researchgate.net*. https://www.researchgate.net/publication/342999107_The_Impact_of_Filter_Size_and_Number_of_Filters_on_Classification_Accuracy_in_CNN.

[12] X. Wu *et al.*, “CTransCNN: Combining transformer and CNN in multilabel medical image classification,” *Knowledge-Based Systems*, vol. 281, pp. 111030–111030, Dec. 2023, doi: <https://doi.org/10.1016/j.knosys.2023.111030>.

[13] “Pytorch vs. Tensorflow: Deep Learning Frameworks 2021 | Built In,” *builtin.com*.
<https://builtin.com/data-science/pytorch-vs-tensorflow>